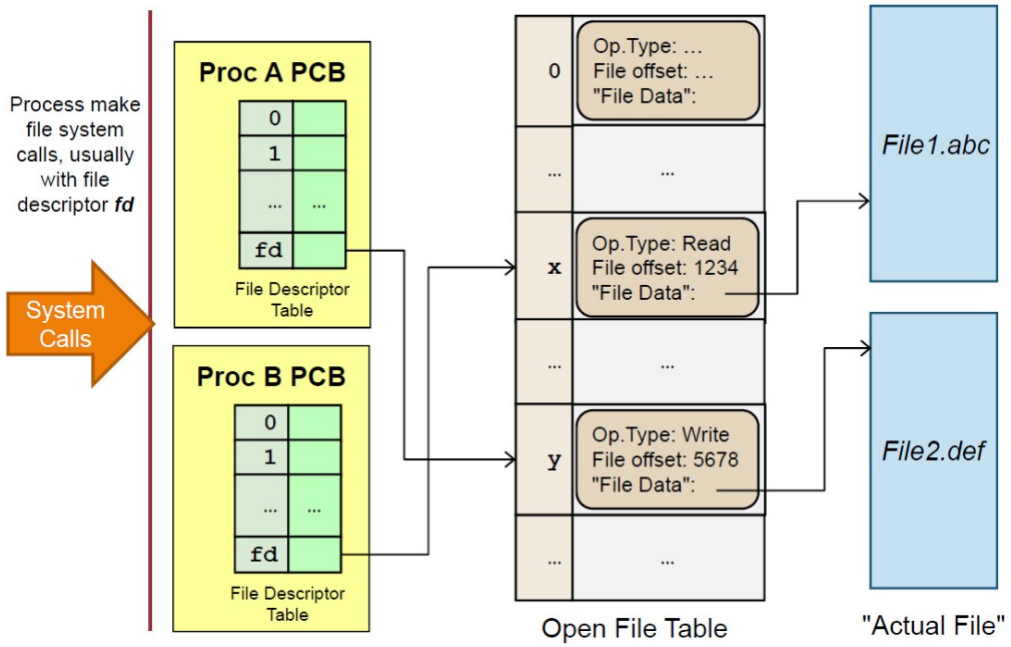
IT5002 Computer Systems and Applications
Tutorial 10

1. [Open File Table] Below is an illustration of how an OS organizes file data at runtime:



Discuss how this organization helps OS to handle the following scenarios. Your answer should refer to the relevant structure(s) if possible.

a. Process A tries to open a file that is currently being written by Process B.

b. Process A tries to use a bogus file descriptor in a file-related system call.

c. Process A can never "accidentally" access files opened by Process B.

d. Process A and Process B reads from the same file. However, their reading should not affect each other.

e. Redirect Process A's standard input / output, e.g. "`a.out < test.in > test.out`". (Hint: *nix processes has 3 default file descriptors: 0 = stdin, 1 = stdout, 2 = stderr)

2. [Putting it together] This question is based on a "simple" file system built from the various components discussed in lecture 12.

Partition Information (Free Space Information)

- Bitmap is used, with a total of 32 file data blocks (1 = Free, 0 = Occupied):

0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**	**15**
0	0	0	0	0	1	0	0	1	0	0	0	0	1	0	1
16	**17**	**18**	**19**	**20**	**21**	**22**	**23**	**24**	**25**	**26**	**27**	**28**	**29**	**30**	**31**
1	0	1	0	1	0	1	0	1	1	1	0	0	1	0	0

Directory Structure + File Information:

- Directory structures are stored in 4 "directory" blocks. Directory entries (both files and subdirectories) of a directory are stored in a single directory block.
- Directory entry:
  - **For File:** Indicates the first and last data block number.
  - **For Subdirectory:** Indicates the directory structure block number that contains the subdirectory's directory entries.
  - The "**/**" root directory has the directory block number 0.

0	**1**	**2**	**3**
y |Dir | 3 f |File| 12| 2 x |Dir | 1	g |File| 0| 31 z |Dir | 2	k |File| 6| 6	i |File| 1| 3 h |File|27|28

File Data:

- Linked list allocation is used. The first value in the data block is the "next" block pointer, with "-1" to indicate the end of data block.
- Each data block is 1 KB. For simplicity, we show only a couple of letters/numbers in each block.

0	**1**	**2**	**3**	**4**	**5**	**6**	**7**
11 AL	9 TH	-1 S!	-1 ND	23 GS	-1 SO	-1 :)	10 TE
8	**9**	**10**	**11**	**12**	**13**	**14**	**15**
31 RE	3 EE	28 M:	31 OH	19 SE	13 AH	4 IN	17 NO
16	**17**	**18**	**19**	**20**	**21**	**22**	**23**
30 YE	2 OU	1 ON	17 RI	26 EV	14 AT	21 DA	7 YS
24	**25**	**26**	**27**	**28**	**29**	**30**	**31**
-1 HO	18 ME	0 AL	30 OP	-1 -(	5 LO	21 ER	-1 A!

a. (Basic Info) Give:
   - The current free capacity of the disk.
   - The current user view of the directory structure.

b. (File Paths) Walkthrough the file path checking for:
   - "**`/y/i`**"
   - "**`/x/z/i`**"

c. (File access) Access the entire content for the following files:
   - "**`/x/z/k`**"
   - "**`/y/h`**"

d. (Create file) Add a new file "**`/y/n`**" with 5 blocks of content. You can assume we always use the free block with the smallest block number. Indicate **all changes required to add the file.**

3. Most OSes perform some higher-level I/O scheduling on top of just trying to minimize hard disk seeking time. For example, one common hard disk I/O scheduling algorithm is described below:

   i. User processes submit file operation requests in the form of
      ***operation*(starting hard disk sector, number of bytes)**
   ii. The OS sorts the requests by hard disk sector.
   iii. The OS merges requests that are nearby into a larger request, e.g. several requests asking for tens of bytes from nearby sectors merged into a request that reads several nearby sectors.
   iv. The OS then issues the processed requests when the hard disk is ready.

   a. How should we decide whether to merge two user requests? Suggest two simple criteria.

   b. Give one advantage of the algorithm as described.

   c. Give one disadvantage of the algorithm and suggest one way to mitigate the issue.

   d. Strangely enough, the OS tends to **intentionally delay** serving user disk I/O requests. Give one reason why this is actually beneficial using the algorithm in this question for illustration.

e. In modern hard disks, algorithms to minimise disk head seek time (e.g. SCAN variants, FCFS etc.) are **built into the** hardware controller. i.e. when multiple requests are received by the hard disk hardware controller, the requests will be reordered to minimise seek time. Briefly explain how the high-level I/O scheduling algorithm described in this question may conflict with the hard disk's built-in scheduling algorithm.